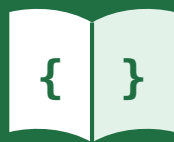

Module 08 — Redis and Caching

Exercises

Spring Boot Edu



2026-09-23

Contents

How to Work	2
Exercise 1 — A Product Detail Cache (Easy)	2
Exercise 2 — A Game Leaderboard (Medium)	3
Exercise 3 — A Rate Limiter (Hard)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/08-redis-caching/exercise/`. Find the `TODO` comments.
2. The tests run against a real Redis (Testcontainers). **Docker must be running.**
3. Run the tests:

```
./mvnw -Pexercises -pl modules/08-redis-caching/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/08-redis-caching/solution/`.

★ Tip

The exercises are independent of each other. Every test class starts from an empty cache or deletes its own keys.

Exercise 1 — A Product Detail Cache (Easy)

Goal: serve product details from Redis and never show an old price after an update.

`ProductCatalog` is given. It is slow (200 ms per read) and counts its reads, so the tests can see whether the cache was used.

Tasks (package `exercisel`):

- `TODO 1a` — `ProductDetailService.detail`: cache the result in the product-details cache.
- `TODO 1b` — `ProductDetailService.changePrice`: after the change, remove **only this product's** entry from the cache.
- `TODO 1c` — `CacheConfiguration`: the product-details cache has a 5 minute TTL, stores JSON typed to `ProductDetail` and does not store null values.

Hints:

- Lesson section 3.1: `@Cacheable(cacheNames = "...")`.
- Lesson section 3.3: `@CacheEvict(cacheNames = "...", key = "#id")`. Without key, the key would be built from **both** parameters and would not match.
- Lesson section 3.2: chain `.withCacheConfiguration("product-details", RedisCacheConfiguration.defaultCacheConfig())` onto the given builder.

Acceptance criteria: all 4 tests in `ExerciselTest` pass.

Estimated time: 25 minutes

Exercise 2 — A Game Leaderboard (Medium)

Goal: keep a live ranking of players in a Redis sorted set.

Tasks (exercise2.Leaderboard):

- TODO 2a — addPoints: add the points to the player's score. A new player starts at 0.
- TODO 2b — top: the best count players, highest score first, as PlayerScore records.
- TODO 2c — rankOf: the player's position: 1 for the leader, 2 for the runner-up, and so on. Empty for a player without points.

Hints:

- Lesson section 3.4: `opsForZSet().incrementScore(...)` and `reverseRangeWithScores(key, 0, count - 1)`.
- `opsForZSet().reverseRank(key, player)` is **0-based** and returns null for an unknown member.
- `Optional.ofNullable(...).map(...)`.

Acceptance criteria: all 3 tests in Exercise2Test pass.

Estimated time: 30 minutes

Exercise 3 — A Rate Limiter (Hard)

Goal: allow each client at most limit calls per time window, for example 100 calls per minute for an API key.

This is a **fixed window** limiter. Time is cut into windows of equal length, and every client has one counter per window. When a new window begins, a new counter begins at 0.

Tasks (exercise3.RateLimiter.tryAcquire):

- TODO 3a — one key per client and window: `rate:<clientId>:<window number>`, where the window number is `clock.millis() / window.toMillis()`.
- TODO 3b — count the call atomically in Redis.
- TODO 3c — the first call of a window makes the key expire after one window, so Redis removes old counters.
- TODO 3d — return true while the count is within the limit.

Hints:

- Lesson section 3.5: `opsForValue().increment(key)` returns the new value. Only the call that gets 1 needs to set the expiry.
- Always take the time from the injected `Clock`, never from `System.currentTimeMillis()`. The test moves the clock forward.
- `redis.expire(key, window)`.

Acceptance criteria: all 4 tests in Exercise3Test pass.

Estimated time: 40 minutes

Extra Challenge (Optional)

A fixed window allows up to twice the limit around a window boundary (the end of one window plus the start of the next). Build a **sliding window** limiter instead: store the timestamp of every call in a sorted set, remove entries older than one window with ZREMRANGEBYSCORE, and count the rest with ZCARD. Why must these steps run in a Lua script or a MULTI/EXEC transaction?